



UNIVERSIDAD DE EL SALVADOR EN LÍNEA
FACULTAD DE INGENIERÍA Y ARQUITECTURA
ESCUELA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS
EDUCACIÓN A DISTANCIA
MICROPROGRAMACIÓN



Visual Studio Code - Raspberry

Objetivo:

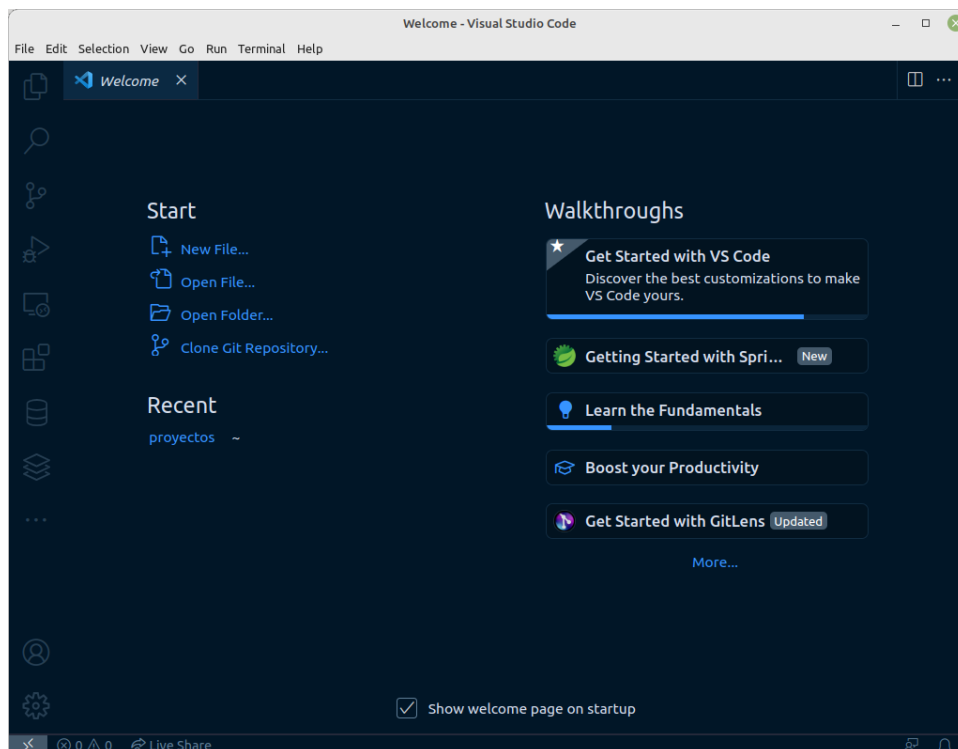
Utilizar el IDE Visual Studio Code (VSC) para editar los programas a utilizar en las prácticas de la asignatura.

Requisitos previos.

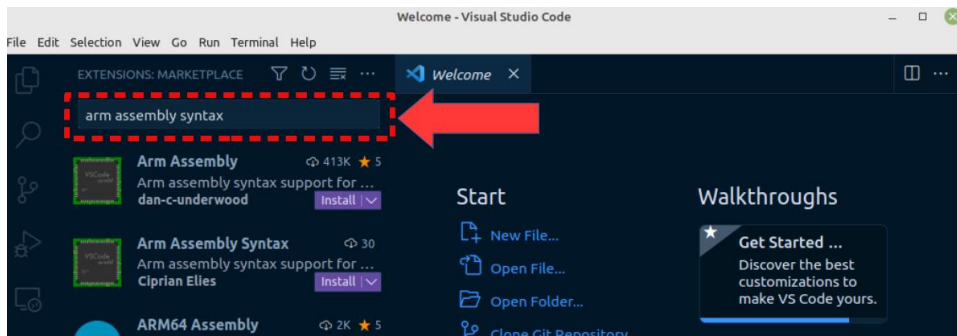
Debe tener instalado VSC, los pasos describirán como configurarlo para trabajar con las prácticas de ensamblador de la asignatura.

Desarrollo de la práctica.

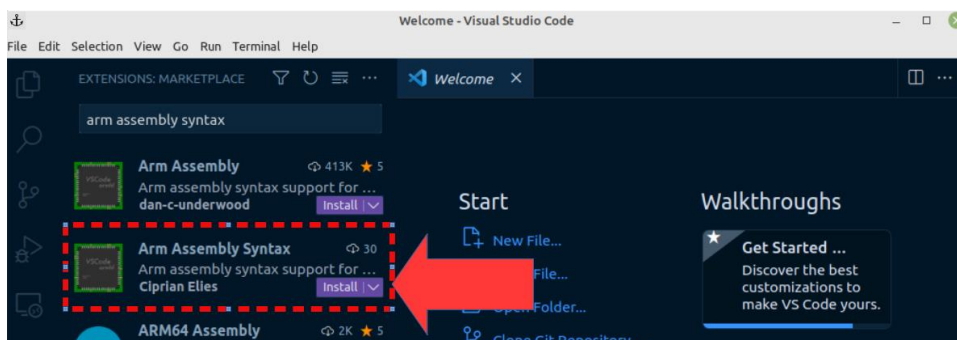
1. Abra VSC



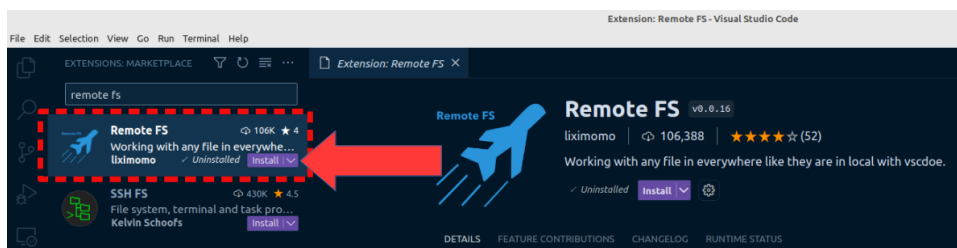
2. Extensión para resaltado de sintaxis de ensamblador ARM.
 - a. Presione ctrl+shift+x para abrir el gestor de extensiones
 - b. En la casilla de búsqueda escriba “arm assembly syntax”



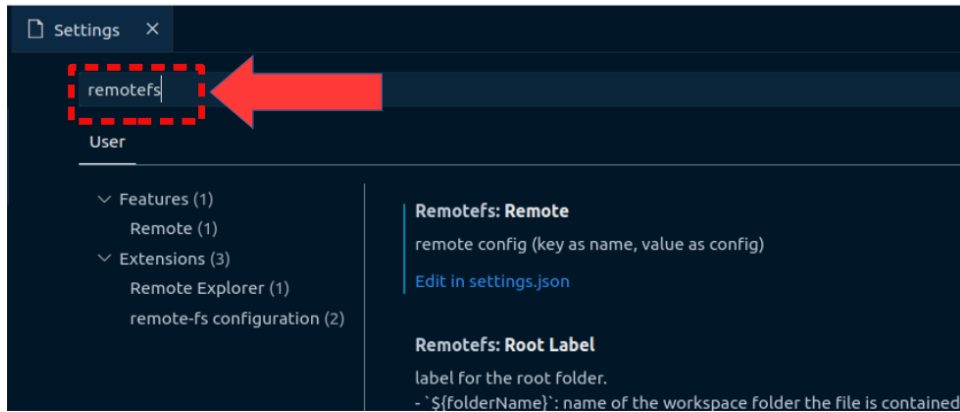
- c. De clic en el botón “Install” de la extensión deseada



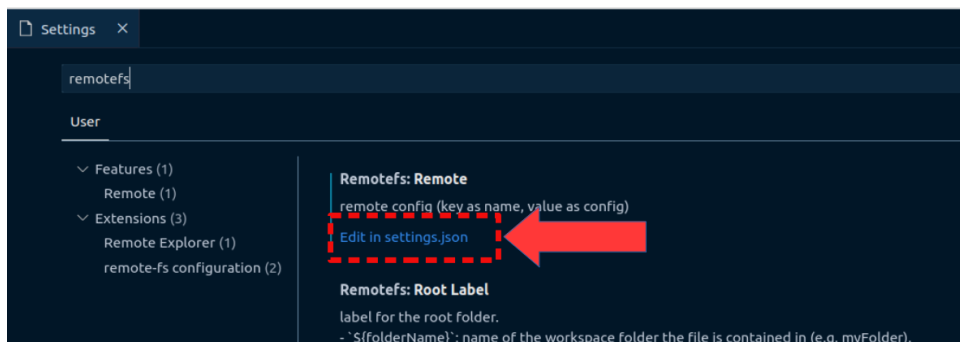
3. Extensión para trabajar de forma remota vía SSH.
 - a. Abra el gestor de extensiones ctrl+shift+x
 - b. En la casilla de búsqueda escriba “remote fs”
 - c. De clic en “install” de la extensión señalada en la imagen



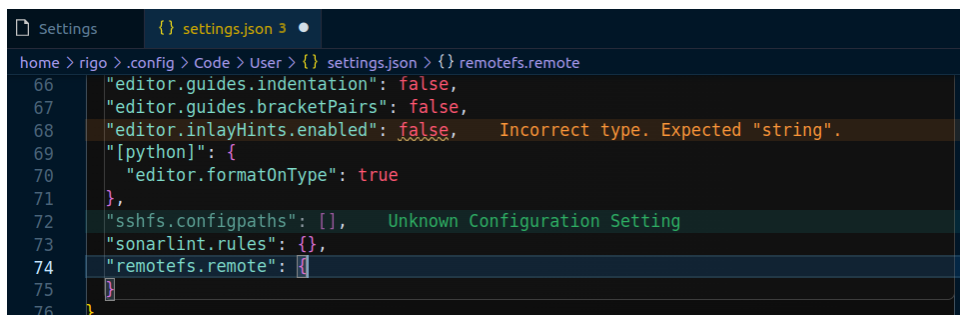
- d. Una vez instalada, vamos a escribir la configuración para realizar la conexión. Presionar Ctrl + , (la tecla control más la tecla de la coma) para abrir la pestaña de configuración.
 - e. Buscamos el texto “remotefs”



f. Luego damos clic en “Edit in settings.json”



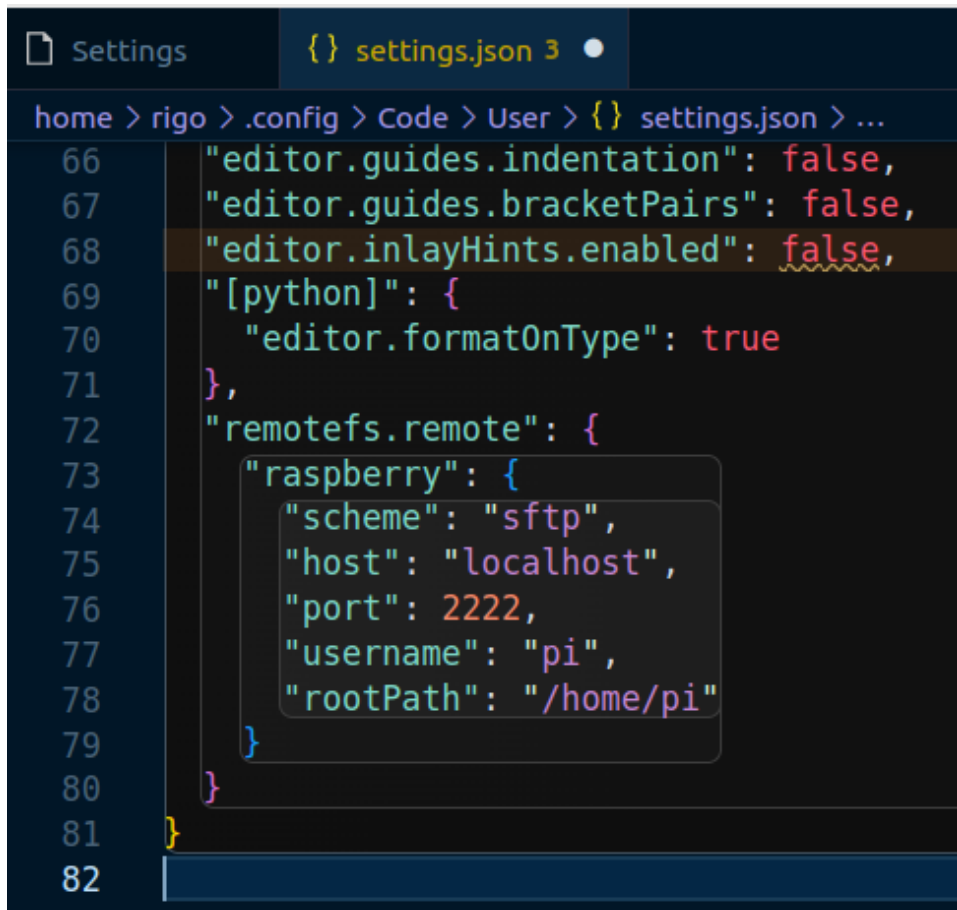
g. Se abrirá un archivo de texto, nos interesa la sección “remotefs.remote”



h. Ahí agregaremos los datos de la conexión. Primero el nombre de la conexión y luego sus parámetros. En este ejemplo la conexión se llama “raspberry”, puede colocar el nombre que desee y luego los parámetros necesarios para conectarnos a la máquina virtual en QEMU.

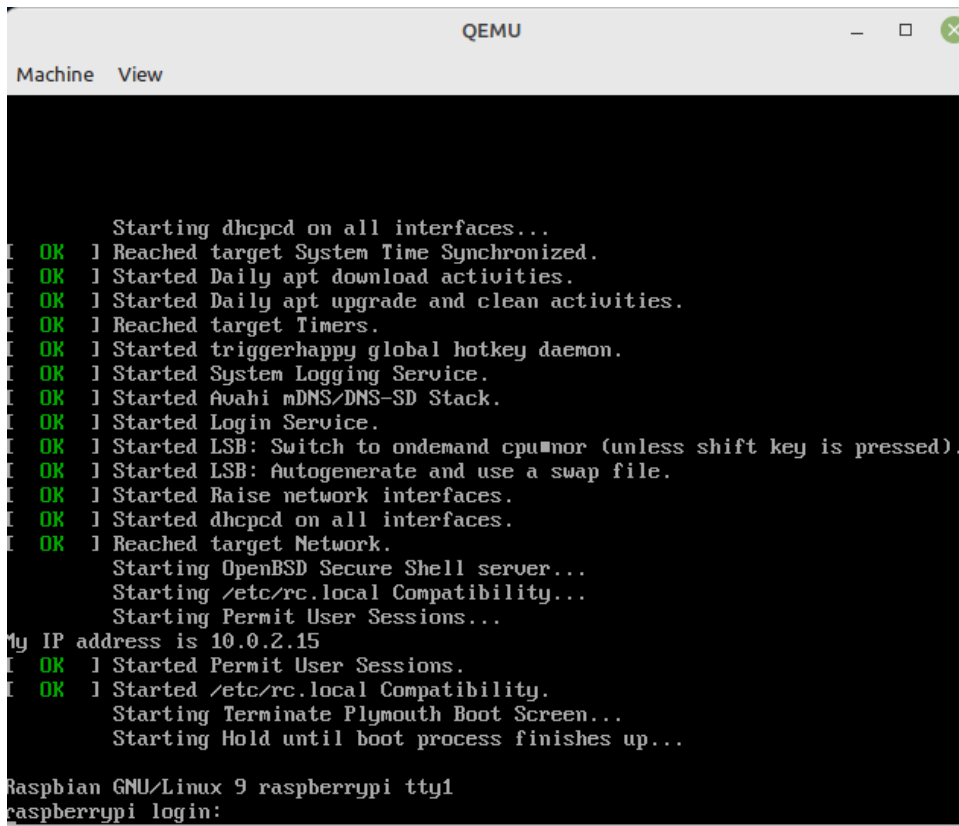
```
"raspberry": {
  "scheme": "sftp",
  "host": "localhost",
  "port": 2222,
  "username": "pi",
  "rootPath": "/home/pi"
}
```

i. El archivo quedará similar a la imagen



```
home > rigo > .config > Code > User > {} settings.json > ...
66 "editor.guides.indentation": false,
67 "editor.guides.bracketPairs": false,
68 "editor.inlayHints.enabled": false,
69 "[python]: {
70   "editor.formatOnType": true
71 },
72 "remoteFs.remote": {
73   "raspberry": {
74     "scheme": "sftp",
75     "host": "localhost",
76     "port": 2222,
77     "username": "pi",
78     "rootPath": "/home/pi"
79   }
80 }
81 }
82
```

- j. Guardamos el archivo presionando ctrl+s
- 4. Realizar la conexión remota.
 - a. Es importante que tengamos la máquina emulada ejecutándose en QEMU. También se asume que ya activó para que se permita la conexión vía SSH, como se describe en la guía de laboratorio #2.

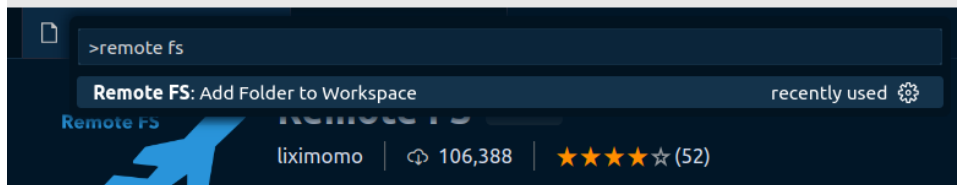


```
Machine View

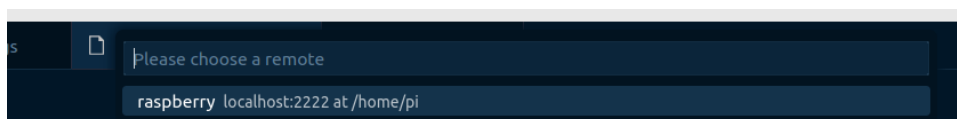
Starting dhcpcd on all interfaces...
OK 1 Reached target System Time Synchronized.
OK 1 Started Daily apt download activities.
OK 1 Started Daily apt upgrade and clean activities.
OK 1 Reached target Timers.
OK 1 Started triggerhappy global hotkey daemon.
OK 1 Started System Logging Service.
OK 1 Started Avahi mDNS/DNS-SD Stack.
OK 1 Started Login Service.
OK 1 Started LSB: Switch to ondemand cpu#nor (unless shift key is pressed).
OK 1 Started LSB: Autogenerate and use a swap file.
OK 1 Started Raise network interfaces.
OK 1 Started dhcpcd on all interfaces.
OK 1 Reached target Network.
Starting OpenBSD Secure Shell server...
Starting /etc/rc.local Compatibility...
Starting Permit User Sessions...
My IP address is 10.0.2.15
OK 1 Started Permit User Sessions.
OK 1 Started /etc/rc.local Compatibility.
Starting Terminate Plymouth Boot Screen...
Starting Hold until boot process finishes up...

Raspbian GNU/Linux 9 raspberrypi tty1
raspberrypi login:
```

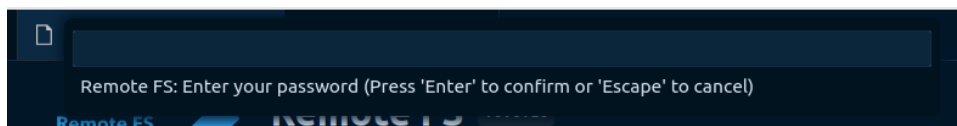
- b. En VSC presionamos ctrl+shift+p y escribimos “remote fs” y elegimos del listado de comandos “Remote FS: Add Folder to Workspace”



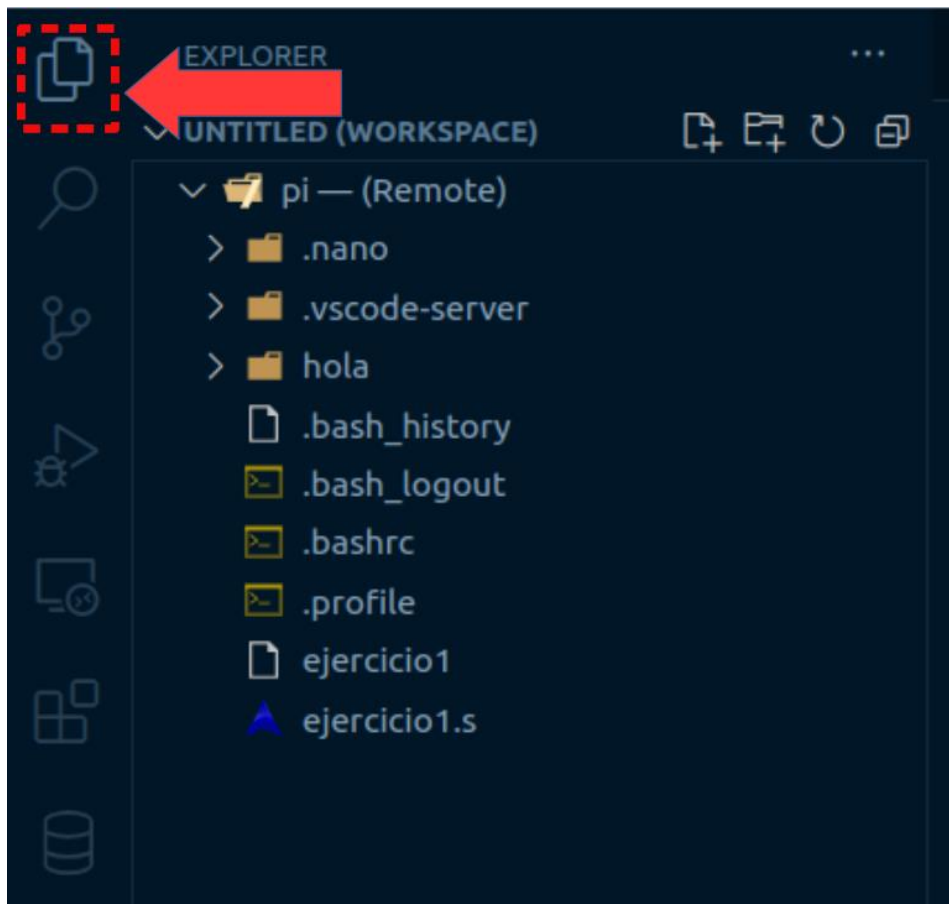
- c. Luego nos pedirá elegir la conexión que vamos a utilizar. Recuerde que hemos creado una conexión llamada “raspberry” la cual elegimos del listado.



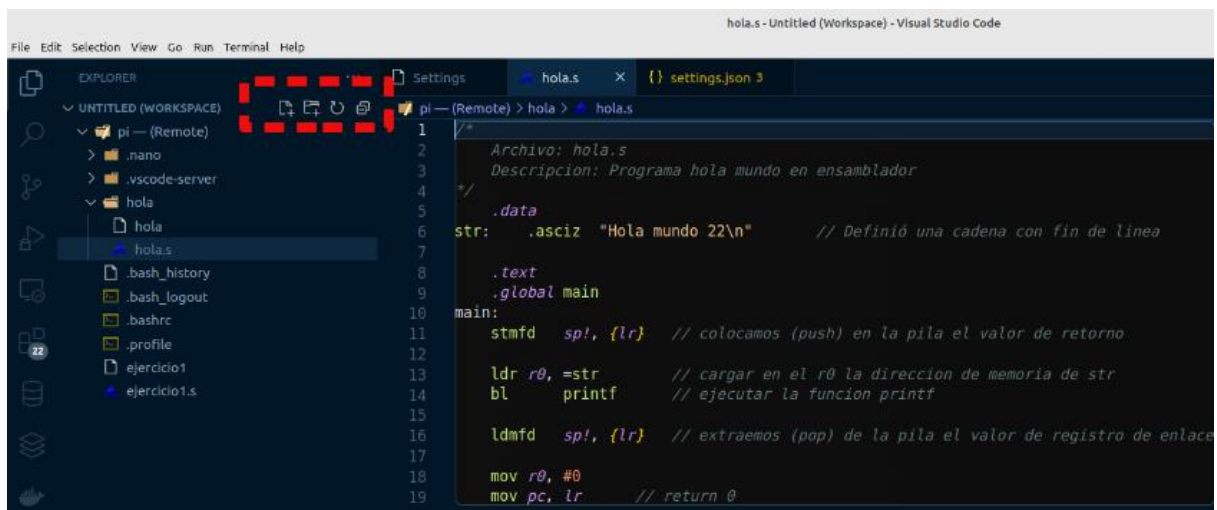
- d. Luego nos pedirá ingresar la clave del usuario “pi”. Si no la ha cambiado es “raspberry”. Presione ENTER después de escribir la clave.



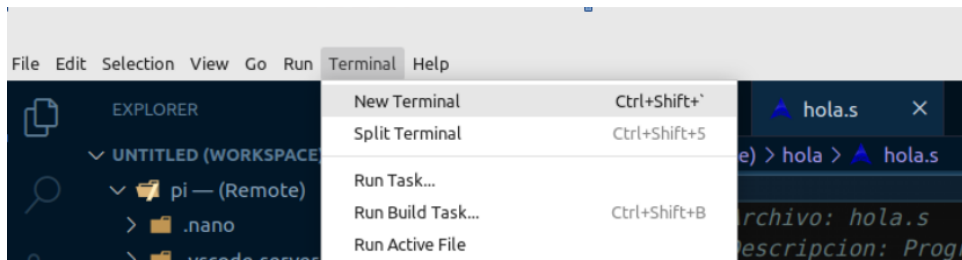
- e. Ahora nos pasamos a la sección del explorador de archivos. En la cual veremos el contenido de la carpeta remota “/home/pi”



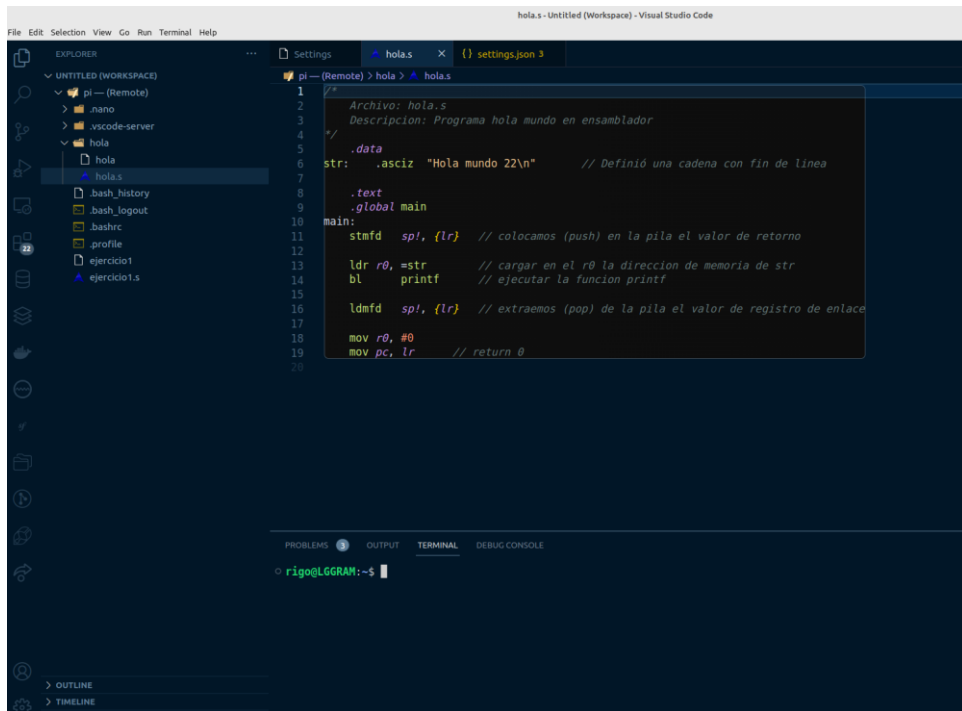
- f. Podemos abrir, o crear nuevos archivos y directorios utilizando los controles que aparecen marcados en la imagen.



- g. Para terminar de configurar el entorno en VSC, abrimos una terminal y nos conectaremos vía ssh, para poder realizar la compilación y prueba de los programas.
- h. En el menú elija Termina > New Terminal.

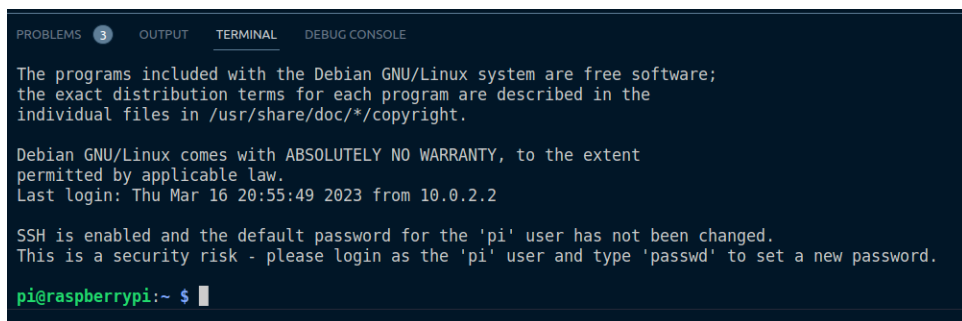


i. Quedará similar a la imagen



j. En la terminal nos conectamos a la raspberry con el comando: `ssh pi@localhost -p 2222`

k. Después de presionar ENTER, se nos pedirá la clave del usuario pi. La escribimos y presionamos ENTER. Si la clave fue correcta se nos mostrará el indicador de comandos de la máquina con raspberry emulada.



l. Desde ahí, entramos hasta donde esté el archivo que deseamos compilar y ejecutamos el comando correspondiente. Por ejemplo:

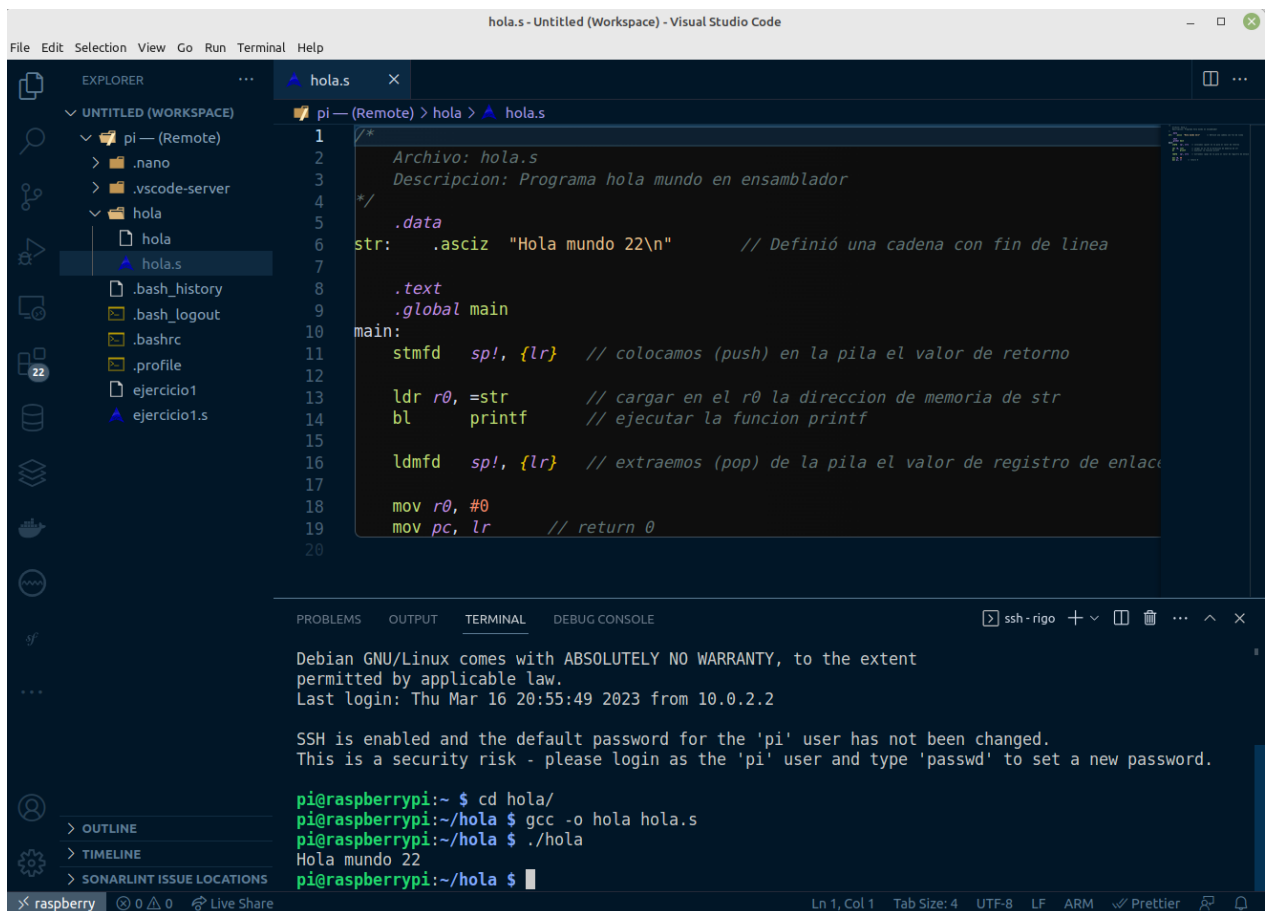
```
PROBLEMS 3 OUTPUT TERMINAL DEBUG CONSOLE

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Mar 16 20:55:49 2023 from 10.0.2.2

SSH is enabled and the default password for the 'pi' user has not been changed.
This is a security risk - please login as the 'pi' user and type 'passwd' to set a new password.

pi@raspberrypi:~ $ cd hola/
pi@raspberrypi:~/hola $ gcc -o hola hola.s
pi@raspberrypi:~/hola $ ./hola
Hola mundo 22
pi@raspberrypi:~/hola $
```

m. Ya tenemos el IDE configurado, en la conexión creamos o abrimos algún archivo, luego lo editamos y en la terminal nos conectamos vía SSH para compilar y ejecutar.



The screenshot shows the Visual Studio Code interface with the 'hola.s' file open in the editor. The file contains assembly code for a program that prints 'Hola mundo 22'. The terminal at the bottom shows the same commands and output as the previous image, confirming the successful compilation and execution of the program.

```
hola.s - Untitled (Workspace) - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXPLORER
  UNTITLED (WORKSPACE)
  pi - (Remote)
    .nano
    .vscode-server
    hola
      hola
      hola.s
      .bash_history
      .bash_logout
      .bashrc
      .profile
      ejercicio1
      ejercicio1.s

hola.s
1 /*
2 Archivo: hola.s
3 Descripcion: Programa hola mundo en ensamblador
4 */
5 .data
6 str: .asciz "Hola mundo 22\n" // Definió una cadena con fin de linea
7
8 .text
9 .global main
10 main:
11 stmfid sp!, {lr} // colocamos (push) en la pila el valor de retorno
12
13 ldr r0, =str // cargar en el r0 la direccion de memoria de str
14 bl printf // ejecutar la funcion printf
15
16 ldmfd sp!, {lr} // extraemos (pop) de la pila el valor de registro de enlace
17
18 mov r0, #0
19 mov pc, lr // return 0
20

TERMINAL
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Mar 16 20:55:49 2023 from 10.0.2.2

SSH is enabled and the default password for the 'pi' user has not been changed.
This is a security risk - please login as the 'pi' user and type 'passwd' to set a new password.

pi@raspberrypi:~ $ cd hola/
pi@raspberrypi:~/hola $ gcc -o hola hola.s
pi@raspberrypi:~/hola $ ./hola
Hola mundo 22
pi@raspberrypi:~/hola $
```